applied sciences

MDPI

Article

Minimizing Hallucinations and Communication Costs: Adversarial Debate and Voting Mechanisms in LLM-Based Multi-Agents

Yi Yang [1], Yitong Ma [1], Hao Feng [1], Yiming Cheng [2] and Zhu Han [3,*]

[1] Department of Electronic Engineering, Tsinghua University, Beijing 100084, China
[2] Department of Computer Science, University of Chicago, Chicago, IL 60637, USA
[3] Department of Computer Science, University of Houston, Houston, TX 77004, USA
* Correspondence: zhan2@uh.edu

Abstract: The emergence of large language models (LLMs), such as GPT and Claude, has revolutionized AI by enabling general and domain-specific natural language tasks. However, hallucinations, characterized by false or inaccurate responses, pose serious limitations, particularly in critical fields like medicine and law, where any compromise in reliability can lead to severe consequences. This paper addresses the hallucination issue by proposing a multi-agent LLM framework, incorporating adversarial and voting mechanisms. Specifically, the framework employs repetitive inquiries and error logs to mitigate hallucinations within single LLMs, while adversarial debates and voting mechanisms enable cross-verification among multiple agents, thereby determining when external knowledge retrieval is necessary. Additionally, an entropy compression technique is introduced to enhance communication efficiency by reducing token usage and task completion time. Experimental results demonstrate that the framework significantly improves accuracy, showing a steady increase in composite accuracy across 20 evaluation batches while reducing hallucinations and optimizing task completion time. Notably, the dynamic weighting mechanism effectively prioritized high-performing models, leading to a reduction in error rates and improved consistency in the final responses.

Keywords: large language models; hallucination reduction; external knowledge retrieval; adversarial debate; voting mechanism; multi-agents

check for updates

Academic Editors: Jose María Alvarez Rodríguez and Chiman Kwan

Received: 30 December 2024
Revised: 24 February 2025
Accepted: 19 March 2025
Published: 27 March 2025

Citation: Yang, Y.; Ma, Y.; Feng, H.; Cheng, Y.; Han, Z. Minimizing Hallucinations and Communication Costs: Adversarial Debate and Voting Mechanisms in LLM-Based Multi-Agents. *Appl. Sci.* **2025**, *15*, 3676. https://doi.org/10.3390/app15073676

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Large language models (LLMs), such as Claude or GPT series [1–3], represent a new era for AI technology. GPT-4 [4], for example, can understand long text inputs, interpret input images, and generate text or image feedback in very complex scenarios. Given the ability to understand complex query intent and solve general natural language tasks, including some domain-specific tasks, some researchers believe that large language models have the potential to evolve into general artificial intelligence [5].

In scenarios such as healthcare and military operations, which require fast and accurate dialogue and environmental responses, LLMs are considered effective and convenient tools [6,7]. However, in these application scenarios, the accuracy of LLMs is crucial, as any errors can lead to serious consequences. For example, in healthcare, an incorrect response could lead to a wrong treatment suggestion, putting the patient at risk; in military operations, misinterpreting information could lead to poor decisions; and in legal cases, wrong information could unfairly influence the outcome.

Hallucinations in LLMs refer to cases where the model automatically generates responses that contain false information or are non-factual [8]. These issues can have serious consequences in scenarios that rely heavily on accuracy. While some efforts have been made to address hallucinations—such as repeated questioning, where the model is asked the same question multiple times, and cross-model comparison, where multiple models with different training backgrounds provide answers—the problem remains far from fully resolved. Existing methods often struggle to generalize across different tasks or provide consistent results, leaving room for improvement in detecting and reducing hallucinations effectively.

In addition to accuracy concerns, dialogue efficiency plays a critical role in LLM applications, especially in time-sensitive or resource-constrained scenarios such as military operations or emergency medical consultations. Excessive dialogue rounds not only lead to increased computational costs but also risk amplifying hallucinations by introducing more opportunities for the model to generate non-factual responses. Therefore, reducing unnecessary dialogue rounds is essential to ensure both operational efficiency and response reliability in LLM interactions.

This paper aims to address the hallucination issue of LLMs by introducing adversarial and voting mechanisms in multi-agent LLMs. Specifically, we propose a multi-agent LLM framework that incorporates repetitive inquiries and error logs within a single LLM, as well as adversarial debates between multiple LLMs. The voting mechanism is embedded in this adversarial debate to determine the necessity of conducting some external knowledge retrieval. Moreover, we employ entropy compression to minimize the number of conversational tokens and turns, in order to handle tasks that are sensitive to communication efficiency. Finally, we use multiple evaluation metrics to evaluate the performance of the proposed method, including the hallucination reduction rate, the total number of conversational tokens, and the task completion time. Our contributions are as follows:

- We propose a multi-agent LLM framework with adversarial and voting mechanisms to detect and reduce hallucinations. In this framework, first, we use repetitive inquiries and error logs to reduce hallucinatory responses for a single LLM. The adversarial debates and voting mechanisms are used to determine whether to retrieve external help to correct the hallucinations.
- We introduce the entropy compression technique to minimize the number of conversational tokens for tasks that are sensitive to communication efficiency.
- We assess the framework's performance by evaluating various metrics, including the hallucination reduction rate, the total number of conversational tokens, and task completion time.

The remainder of this paper is organized as follows. Section 2 presents related works. Section 3 presents our proposed methodology, including the system architecture and key techniques. Section 4 describes the experimental design, implementation details, and results. Finally, Section 5 presents the conclusion.

2. Related Work

2.1. Generative AI Models and Foundation Models

Generative AI models are a class of machine learning models designed to generate new data that closely resemble the distribution of the original data. [9] The initial AI content generation technology gained attention around the year 2015 [10,11]. In addition to generating conversational text, this type of AI technology started to generate multimodal content, such as images [12,13] and videos [14]. Typical methods for generating AI models include the Gaussian mixture model, hidden Markov model, latent Dirichlet allocation, Boltzmann machine, variational autoencoder, generative adversarial network [12], etc.

Foundation models represent a paradigm shift in AI based on the progress of generative models. As defined by Bommasani et al. [15], foundation models are trained on large and extensive amounts of data and are adaptable to a variety of downstream tasks by fine-tuning. Foundation models are seen as incomplete but critical building blocks that are adaptable and serve specific tasks. However, this adaptability may also cause hallucinations when the models deal with unclear or confusing input. This highlights the need for robust mechanisms to detect and reduce hallucinations in foundation models.

2.2. Large Language Models

Large language models have seen rapid advancements, evolving from task-specific fine-tuning models to versatile systems capable of general and domain-specific tasks. In August 2024, the ChatGPT platform (ChatGPT 4o, 4 Turbo,4o with Canvas) had 180.5 million registered users. Microsoft's Bing AI(v28.5.2110003549) had approximately 100 million daily active users. At the same time, Anthropic's Claude AI had millions of daily active users.

The GPT series plays an important role in the area of large language models. In June 2018, the GPT-1 model [1], with 117 million parameters, was introduced. The model was based on the Transformer framework and trained using unsupervised learning methods. Fine-tuning was required to achieve downstream tasks. In February 2019, the GPT-2 model [2], with 1.5 billion parameters, was able to achieve multi-task learning and did not require fine-tuning on some supervised subtasks. GPT-3 [3] had 175 billion parameters; the model had better generalization ability and performed well in various downstream tasks. The InstructGPT (used in GPT3.5) [16] version introduced reinforcement learning with human feedback (RLHF). The public version of ChatGPT was released to the public at the end of 2022; two months later, it reached 100 million monthly active users. GPT-4 [4], released in March 2023, has evolved into a large multimodal model that can accept image and text inputs. GPT-4 performs at the same level as humans on many benchmarks [17].

2.3. Hallucination Problem in Large Language Models

Hallucinations [18] in large language models (LLMs) refer to cases where the models generate incorrect or meaningless outputs. Researchers have categorized hallucinations into the following six types: acronym ambiguity, numeric nuisance, generated golem, virtual voice, geographic erratum, and time warp [19]. Although existing research has explored various techniques to detect and mitigate these hallucinations, challenges remain due to the complexity of LLMs and their training data.

Hallucinations originate from three stages, namely, data collection, training, and inference. First, in the data collection stage, the training data may be of low quality and inconsistent. There may be erroneous and outdated data, and there may also be potential bias in the training data. In the training stage, the attention mechanism in the Transformer architecture cannot handle sequences that are too long. In the inference stage, there are differences between the training data and the inference data, and the randomness of sampling leads to the generation of meaningless response content. These are all possible causes of hallucinations [18].

Two types of hallucinations can potentially be triggered in large language models that incorporate external knowledge, such as knowledge graphs, namely, extrinsic hallucinations and intrinsic hallucinations. The former is easier to detect, characterized by the model responding with facts that do not exist in the knowledge graph, and its induction is mainly due to the system's text spans; the latter is characterized by the model making incorrect references to content in the knowledge graph, and its induction is mainly due to the system's incorrect associations [20].

2.4. Approaches to Deal with the Hallucination Problem

Detecting hallucinations in LLM to enhance model performance is crucial to avoid misinformation, ensure factual consistency, and protect user trust and information security [21]. Currently, there are some datasets dedicated to the hallucination phenomenon in LLMs, such as HaluEval Datasets [22].

As LLMs become prevalent in critical fields, such as education, scientific research, and the military, the accuracy of LLM responses is more paramount. It is important to propose more effective strategies for mitigating LLM hallucinations. In the latest LLM hallucination mitigation research, several solutions have been proposed, including identifying high-entropy words in generated text and replacing them with words that have a lower (HVI); scoring sentences in generated text and conducting manual reviews of low-scoring sentences; reranking and beam search improvements, etc. [19].

In addition to reducing the toxicity of hallucinations in individual LLMs, the use of multiple LLM agents has also begun to receive attention due to their effectiveness in mitigating hallucinations. The term "multi-agent" in LLMs refers to multiple LLM agents, which are employed to debate with each other to solve complex tasks. When multiple agents are asked to respond to the same prompt—if one agent experiences hallucinations, other agents can cross-verify the response information and provide alternative responses. And, the degeneration-of-thought (DoT) problem occurs when the single LLM is confident in its initial answer and cannot generate new ideas through self-reflection, even if the initial answer is incorrect. By having multiple agents with different perspectives, this architecture could challenge the perspectives of other agents and attempt to avoid DoT [23,24].

2.5. System Optimization for Reducing Dialogue Rounds

The optimization of dialogue systems to minimize the number of interaction rounds has received some attention in the field of natural language processing (NLP). Although not a dominant research area, efforts in this direction aim to enhance efficiency by reducing redundant conversational turns while complementing the primary goal of generating coherent and contextually relevant responses.

One line of research explores reinforcement learning (RL) for dialogue policy optimization. For instance, studies such as those leveraging deep Q-networks (DQNs) [25] aim to identify optimal strategies for task-oriented dialogues. These approaches reward concise and effective interactions, guiding the system to achieve the desired outcome in fewer turns [26,27]. Reinforcement learning frameworks often incorporate reward shaping to penalize unnecessary dialogue steps, thereby fostering more efficient exchanges. However, applying RL to modern large-scale language models like GPT and BERT-based architectures is challenging due to the substantial computational resources required and the complexity of training such models. The high cost of reinforcement learning optimization in these advanced systems highlights the need for more efficient methodologies.

Another strategy focuses on improving dialogue state tracking and response generation. Enhanced state tracking mechanisms allow the system to maintain a more accurate representation of user intent and conversation history, reducing errors and misunderstandings that can lead to prolonged interactions [28]. Meanwhile, advancements in pre-trained language models, such as GPT and BERT-based architectures, enable better contextual understanding and response formulation, indirectly contributing to fewer dialogue rounds [3,29]. However, for scenarios where computational resources are limited, optimizing existing model architectures rather than pursuing entirely new and resource-intensive designs remains a highly valuable approach.

3. Methodology

3.1. System Architecture

The proposed system integrates multiple complementary components to detect and reduce hallucinations in large language models (LLMs), as shown in Figure 1. The algorithm is shown in Algorithm 1. At its core, the architecture relies on a collaborative framework where multiple LLMs operate in tandem to evaluate and validate responses. Each LLM independently engages in repetitive inquiries by answering the same query multiple times, aiming to identify inconsistencies and potential hallucinations through repetitive questioning.

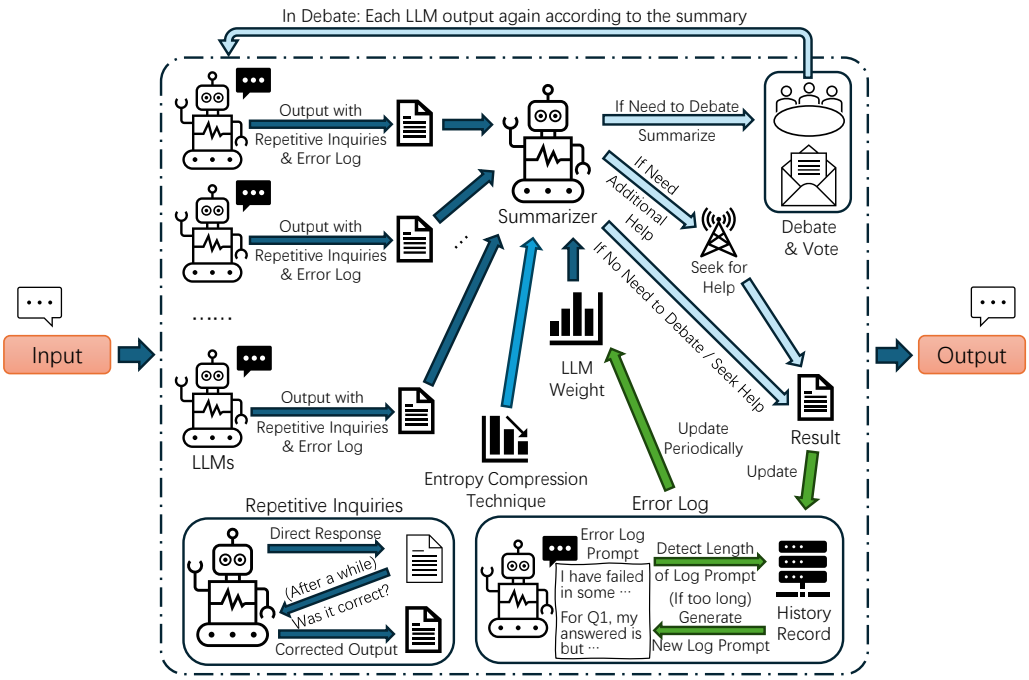

Figure 1. System architecture.

A multi-agent adversarial mechanism further enhances the system's robustness. Different LLMs, trained on diverse datasets, dynamically interact to cross-validate their outputs. To ensure unbiased and reliable responses, the system employs a reliability-based weighting mechanism that adjusts each model's contribution based on the reliability metrics. This adversarial setup leverages the unique strengths of each LLM, reducing vulnerability to individual model biases.

Error tracking is incorporated into each LLM through an error log, which records instances of inconsistencies. When the error count exceeds a predefined threshold, a self-reflection process is triggered, enabling the LLM to review its past mistakes and refine future responses. This mechanism fosters continuous improvement within individual models.

To combine and check the outputs of different LLMs, a GPT-based summarizing model is employed. This module combines the responses from multiple models and performs additional validation. Hallucination detection is further reinforced by analyzing the variance and entropy of the generated responses. If hallucinations are identified, the system either internally corrects the response or refers it to a stronger model or human evaluator. Finally, a penalty function discourages excessive response variance and minimizes unnecessary dialogue rounds, ensuring both accuracy and efficiency.

Algorithm 1 Hallucination detection, reduction, and final response generation.

```
Input: Query Q
Output: Corrected response R_final
Initialize models LLM_1, LLM_2, ..., LLM_N // Multiple LLMs
Initialize summarizing GPT model GPT_summary // Logs the prompts each LLM used
Initialize error logs for each LLM_i
Set threshold T_error for self-reflection
Set threshold T_variance for hallucination detection
// Repeat until stopping criteria are met
while stopping criteria are not met do
  // Multi-Agent Query Processing
  for each LLM_i do
    Generate initial response R_i^(0) ← LLM_i(Q)
    Track error for R_i^(0) in error log ErrorLog_i
    // Repetitive Inquiries within LLM_i
    for r = 1 to N (dialogue rounds) do
      Use the error log ErrorLog_i to generate the corresponding prompt
      Generate new response R_i^(r) ← LLM_i(Q)
      Calculate variance σ_i^2 between R_i^(r) and previous responses
      if σ_i^2 > T_variance then
        Flag R_i^(r) as potential hallucination
      end if
    end for
    if error count in ErrorLog_i > T_error then
      Trigger self-reflection for LLM_i
      Update internal parameters of LLM_i to avoid future errors
    end if
  end for
  // Multi-agent voting
  Aggregate responses from all LLM_i and generate chain-of-thought (CoT)
  for each LLM_i in all agents do
    Perform voting based on R_i and chain-of-thought
    if conflicting responses between agents then
      Initiate voting process
      Adjust weights of models based on their reliability using softmax
    end if
  end for
  // Summarizing response
  Aggregate all responses from LLM_1, LLM_2, ..., LLM_N
  Perform final verification of aggregated responses with GPT_summary
  If inconsistencies are found, correct or refer for further evaluation
  // Final response check
  if the aggregated response passes all validation checks (low variance, consistent
  reasoning, number of rounds) then
    Exit loop and finalize the response
  end if
end while
Output: R_final
```

3.2. Single LLM Processing

In the single LLM processing phase, the system employs two key strategies to enhance the reliability and accuracy of responses generated by each individual language model (LLM). The first strategy involves repetitive inquiries within a single LLM, where the LLM generates multiple responses to the same question over several dialogue rounds.

This approach allows the system to assess the consistency of the LLM outputs and detect potential hallucinations—discrepancies that suggest instability in the model's reasoning.

The second strategy involves an error log and self-reflection mechanism, where each LLM records and analyzes its past mistakes. When a model's error log reaches a certain threshold, the LLM reviews its previous errors and adjusts its response strategy by modifying prompts to prevent similar mistakes from recurring. By dynamically refining its own prompts based on historical errors, the LLM can improve its future performance and reduce inconsistencies in its outputs.

Together, these strategies help the LLMs autonomously assess their responses, self-correct, and refine their behavior, ultimately reducing the likelihood of hallucinations and enhancing the overall accuracy and reliability of the system.

3.2.1. Repetitive Inquiries Within Single LLM

The process of repetitive inquiries within a single LLM plays a key role in hallucination detection by enabling the LLM to self-assess the consistency of its responses. In this process, each LLM autonomously evaluates its own responses by engaging in repetitive inquiries within a single LLM on the same question. For instance, when presented with a task-specific question such as "Fetch a glass of water", the LLM generates an initial response based on its current state. This question is then reintroduced in multiple dialogue rounds, where the LLM is prompted to generate responses anew each time.

The purpose of these repeated rounds is to assess the consistency of the LLM's outputs. By comparing the responses across rounds, the system can calculate the variance between them. The high variance between responses signals potential hallucinations, indicating that the LLM's understanding or reasoning might be inconsistent or flawed. If the variance exceeds a predefined threshold, the response is flagged for further evaluation. This iterative process not only identifies potential hallucinations but also triggers corrective mechanisms, such as referencing internal error logs or consulting a more robust model for verification.

The effectiveness of this repetitive inquiry approach is measured by the hallucination reduction rate, which quantifies the system's success in minimizing inconsistent or incorrect outputs. By integrating this self-assessment process, the LLMs are empowered to refine their own responses, thereby reducing the likelihood of hallucinations in practical applications.

3.2.2. Error Log and Self-Reflection

The error log and self-reflection mechanism are essential components of the system's ability to learn from past mistakes and improve performance. Each LLM maintains an error log that records instances where its responses are flagged as incorrect or inconsistent. This log serves as a memory bank of the model's past errors, allowing it to track recurring mistakes and identify patterns over time.

When a model's error log reaches a predefined threshold, a self-reflection process is triggered. In this process, the LLM reviews its past errors and uses this information to adjust its response strategy. Specifically, the model generates a new prompt based on its previous mistakes, using the error log to guide the formulation of questions or tasks. For instance, if a particular type of error occurs repeatedly, the LLM can adjust the prompt to clarify ambiguities or reframe the question in a way that minimizes the chance of repeating the same mistake.

This adjustment is not just a passive review but actively influences the model's reasoning process. By tailoring prompts based on prior errors, the LLM can refine its decision-making, reduce inconsistencies, and avoid hallucinations in future responses.

Moreover, this self-reflection mechanism is integrated with the broader system. For example, when a new query is presented, the system references the error log to check if

the current situation resembles any past mistakes. If a similar error pattern is detected, the system can modify the prompt accordingly before generating a response.

The effectiveness of this mechanism is measured by improvements in the consistency and accuracy of the LLM's responses, as well as a reduction in hallucinations over time. By continuously learning from its mistakes and adjusting the prompt dynamically, the LLM becomes more resilient to errors and can generate more accurate and reliable outputs.

3.3. Multi-Agent Processing

This section outlines the multi-agent processing approach, where multiple LLMs collaborate to improve response accuracy and reduce hallucinations. The multi-agent adversarial mechanism enables models to challenge each other's outputs, fostering consistency. The summarizing GPT model aggregates and evaluates these responses, correcting any inconsistencies and ensuring robustness. The multi-agent voting mechanism resolves disagreements among the models by using chain-of-thought reasoning, allowing for a more reliable final decision. Together, these components work in tandem to optimize the system's performance and enhance response quality.

3.3.1. Multi-Agent Adversarial Mechanism

The multi-agent adversarial mechanism is a key component in our system, designed to leverage the diversity and strengths of multiple large language models. Each LLM in the system is pre-trained on different datasets, introducing varied perspectives and approaches to problem-solving. This diversity is harnessed through an adversarial framework, where the models engage in a dynamic interaction to validate and challenge each other's responses.

In this mechanism, the responses from each LLM are compared, and their reliability is evaluated based on the consistency and correctness of their outputs. A softmax function is employed to dynamically adjust the weight of each LLM's contribution to the final decision, with models that consistently provide accurate responses being given higher weight. This process allows the system to balance the input from multiple models, reducing the influence of any single model that may exhibit hallucinations or biases.

The adversarial setup not only enhances the accuracy of the system but also contributes to reducing the occurrence of hallucinations. By having models effectively "debate" their outputs, the system can identify inconsistencies and flag potential errors before a final response is generated. This mechanism ensures that the combined strengths of the LLMs lead to more reliable outputs, ultimately improving the overall performance of the system.

3.3.2. Summarizing GPT Model

The summarizing GPT model serves as the central coordinator in our system, synthesizing the outputs from multiple LLMs and guiding the dialogue process. Unlike the individual LLMs that focus on generating responses based on their training, the summarizing GPT model is responsible for evaluating the coherence and consistency of the collective outputs. It achieves this by aggregating the responses from all participating LLMs and identifying potential inconsistencies or hallucinations.

One of the key roles of the summarizing GPT model is to introduce new questions into the dialogue that are designed to test the robustness of the LLMs. These questions often contain subtle errors or ambiguities that challenge the LLMs' reasoning capabilities. By embedding these potential pitfalls, the summarizing GPT model ensures that the LLMs are not merely generating surface-level responses but are critically evaluating the questions and their own outputs.

Additionally, the summarizing GPT model has access to the error logs from each LLM, which allows it to reference past mistakes during the dialogue process. If a current response

closely resembles a previously flagged error, the summarizing GPT model can prompt further verification or corrections before finalizing the response. This proactive approach helps to minimize the likelihood of repeated mistakes and enhances the overall accuracy of the system.

The effectiveness of the summarizing GPT model is evaluated through metrics such as the reduction in human–machine interaction and the overall task completion time. By efficiently guiding the dialogue process and ensuring accurate responses, the summarizing GPT model plays a crucial role in optimizing the system's performance.

3.3.3. Multi-Agent Voting Mechanisms

Since ancient Greece, debate has been used as a method to refine opinions and make informed decisions. Analogous to multi-person debates, multi-agent LLMs could achieve collaboration by forming LLM communities and debating each other [30]. To reduce the interference of stereotyping or pre-trained knowledge, we propose multi-agent voting mechanisms, that is, each agent (LLM) is set a priori as a participant with different preferences, and votes independently on whether the response of a single LLM is a hallucination after a debate occurs. For example, we have the following:

> "You are a robot responsible for providing home services to users. When making decisions, your first criterion is to protect the user's physical safety. You are wary of unfamiliar objects and usually deal with them by moving them outdoors."

Multiple voting is required when two or more agents disagree about the same event. For example, a user complains of a headache after breathing in cooking fumes. Single agents (LLMs) may generate content suggesting that headaches, in such cases, are often associated with carbon monoxide poisoning. Therefore, multiple agents have to vote on whether to contact emergency services.

Furthermore, we incorporate chain-of-thought (CoT) reasoning into each agent before voting to provide more references for other agents. For example, consider a scenario where a gas leak is detected in the home, and the group needs to vote on whether to contact emergency services. The following is the CoT of agent 1 before voting:

- Confirmed through multiple sensors that a gas leak has occurred and the user is at home.
- There is an immediate and serious danger to the safety of the user due to the risk of explosion or poisoning.
- Confirmed that the gas leak cannot be repaired and that the user cannot be evacuated safely.
- Data indicate a serious hazard, and failure to act could result in serious harm to the user.
- Vote in favor of contacting emergency services immediately.

3.4. System Optimization

System optimization is crucial to ensure that the proposed framework operates efficiently and effectively, minimizing unnecessary computations while maintaining high accuracy in hallucination detection. The optimization process focuses on balancing various factors, including reducing dialogue rounds, minimizing response variance, and ensuring timely task completion.

One of the key objectives in system optimization is to find the optimal trade-off between accuracy and efficiency. This can be formulated as a multi-objective optimization problem, where we aim to minimize both the total response time $T_{response}$ and the number of tokens used T_{total}, while also minimizing hallucination occurrence. The optimization problem can be expressed as follows:

$$\min_{W}\left(\lambda_1 T_{response}(W) + \lambda_2 T_{total}(W) + \lambda_3 H_{rate}(W)\right), \tag{1}$$

where W represents the model parameters, λ_1, λ_2, and λ_3 are weighting factors that balance the importance of each term, and H_{rate} is the hallucination rate, defined as the percentage of responses that are flagged as hallucinations.

3.4.1. Variance Penalty for Dialogue Rounds

Reducing the number of dialogue rounds is essential for optimizing the efficiency of the system, particularly in real-time applications where response time is critical. To achieve this, we introduce a mathematical approach that minimizes unnecessary dialogue rounds while maintaining accuracy in the responses.

The core idea is to penalize response variance across dialogue rounds, thereby encouraging the LLMs to converge on consistent answers more quickly. Let R_i represent the response generated by the LLM in the i-th round, and μ represent the mean response across all rounds. The variance σ^2 of the responses can be calculated as follows:

$$\sigma^2 = \frac{1}{N}\sum_{i=1}^{N}(R_i - \mu)^2, \tag{2}$$

where N is the total number of dialogue rounds. A higher variance indicates a greater inconsistency in the LLM's responses, which is a potential sign of hallucination. To reduce the variance and, consequently, the number of rounds, we introduce a penalty function P based on the variance:

$$P = \alpha \cdot \sigma^2, \tag{3}$$

where α is a scaling factor that adjusts the weight of the penalty. This penalty is added to the system's loss function during training, encouraging the LLM to generate consistent responses with minimal rounds.

3.4.2. Entropy Minimization for Dialogue Efficiency

In addition to variance reduction, another approach to minimize dialogue rounds involves controlling the entropy of the responses. Entropy H quantifies the uncertainty of the LLM's responses, and is defined as follows:

$$H = -\sum_{k} p_k \log p_k, \tag{4}$$

where p_k is the probability of the k-th response option. High entropy suggests that the LLM is uncertain about its responses, which could lead to prolonged dialogues. By minimizing entropy alongside variance, we ensure that the LLMs provide more confident and consistent answers, thus reducing the need for additional rounds.

Finally, the system monitors the combined effect of these measures by evaluating the total number of tokens used across all rounds T_{total}, and the total response time $T_{response}$. The objective is to minimize the following:

$$\min(T_{total}, T_{response}). \tag{5}$$

By integrating variance and entropy penalties into the dialogue process, the system efficiently reduces the number of dialogue rounds while maintaining response quality, leading to faster and more accurate task completion.

3.4.3. Gradient-Based Optimization

To solve the multi-objective optimization problem, we employ a gradient-based optimization approach. The gradient of the objective function with respect to the model parameters W is given by the following:

$$\nabla_W \mathcal{L}(W) = \lambda_1 \nabla_W T_{response}(W) + \lambda_2 \nabla_W T_{total}(W) + \lambda_3 \nabla_W H_{rate}(W). \tag{6}$$

By iteratively updating the model parameters using a gradient descent algorithm, we can minimize the objective function:

$$W^{(t+1)} = W^{(t)} - \eta \nabla_W \mathcal{L}(W), \tag{7}$$

where η is the learning rate, and t represents the iteration step.

3.4.4. Adaptive Penalty Function

In addition to gradient-based optimization, we introduce an adaptive penalty function to dynamically adjust the system's behavior based on real-time performance metrics. The penalty function P is designed to penalize high variance σ^2 and entropy H in the LLMs' responses, as discussed earlier. The adaptive penalty function can be formulated as follows:

$$P(W) = \beta_1 \sigma^2(W) + \beta_2 H(W), \tag{8}$$

where β_1 and β_2 are adaptive weights that are updated based on the system's current performance. For example, if the system detects a high hallucination rate, β_1 and β_2 can be increased to penalize variance and entropy more heavily, encouraging the models to produce more consistent and confident responses, ultimately reducing hallucinations.

The adaptive nature of this penalty function enables the system to adjust to different tasks and scenarios, ensuring that the optimization process remains effective in a wide range of conditions.

3.4.5. Regularization

To further enhance system robustness, we apply regularization techniques that prevent overfitting and ensure generalization across different tasks. The regularization term $R(W)$ is added to the objective function:

$$\mathcal{L}_{total}(W) = \mathcal{L}(W) + \lambda_4 R(W), \tag{9}$$

where $R(W)$ could represent L_2 regularization (ridge regression), defined as follows:

$$R(W) = \frac{1}{2} \|W\|^2. \tag{10}$$

This regularization term helps in constraining the model parameters, preventing them from becoming too large and overfitting the training data.

4. Experiments

This section describes the experimental setup, the comparison with the baseline method, and the ablation study conducted to evaluate the proposed multi-agent LLM framework.

4.1. Experimental Setup

The experiments were designed to evaluate the proposed multi-agent LLM framework for reducing hallucinations and optimizing dialogue efficiency. The setup includes task scenarios, datasets, models, a hardware environment, and an experimental codebase.

4.1.1. Task Scenario

The task scenario involves solving mathematical problems and general knowledge questions to assess hallucination detection and reduction. Tasks include randomly generated arithmetic problems and questions from established benchmarks [5].

Datasets

- Baseline datasets:
  - MIT MMLU dataset: A benchmark for multi-task language understanding, covering diverse domains [5].
  - Mathematics dataset: Randomly generated arithmetic problems (addition, subtraction, multiplication, and division) using custom Python scripts.
- Enhanced datasets:
  - HaluEval dataset: Designed for hallucination evaluation, sourced from the HaluEval GitHub repository. This complements MMLU by targeting hallucination-specific scenarios [31].

4.1.2. Models

- Baseline models:
  - MIT group:
    - Model 1: `orca`
    - Model 2: `wizardlm`
    - Model 3: `vicuna`
    - Summary model: `ChatGPT-3.5` (via Hugging Face API) [3].
  - Domestic group:
    - Model 1: `qwen2-7b`
    - Model 2: `longalign-7b-64k-base-mlq`
    - Model 3: `yi-6b`
    - Summary model: `ChatGPT-3.5` (via Hugging Face API) [3].
- Enhanced experiment models:
  - Accounts/fireworks models:
    - Model 1: `llama-v3p1-8b-instruct`
    - Model 2: `mixtral-8x7b-instruct`
    - Model 3: `phi-3-vision-128k-instruct`
    - Summary Model: `gemma` (via Together.ai API).

4.1.3. Hardware Environment

The experiments were conducted on a system with the following specifications:

- GPU: NVIDIA A40 GPUs (2 units), each with 46 GB of memory.
- Driver and CUDA:
  - Driver Version: `555.42.06`
  - CUDA Version: `12.5`
- Memory and utilization:

- Total GPU memory: `46,068 MiB` (per GPU)
- Idle GPU utilization during baseline testing.

4.1.4. Experimental Codebase

The baseline experiments were implemented using the open-source repository https://github.com/gauss5930/LLM-Agora (accessed on 1 January 2025). Modifications were made to incorporate custom datasets and evaluation metrics [3,29].

4.2. Experimental Steps

This section outlines the experimental steps of our study. The first part describes the baseline setup based on LLM Agora, followed by our extensions and improvements.

4.2.1. Baseline: LLM Agora

Our study builds on the *LLM Agora* framework, which applies the multi-agent debate method introduced by Du et al. [32]. This framework focuses on improving the quality of responses from open-source language models (LLMs) through iterative debates. The main steps are as follows:

Step 1: Initial response generation

Multiple open-source LLMs, such as Llama2, Vicuna2, and WizardLM2, are used to generate responses independently for the following tasks:

- Math problems: Simple arithmetic operations involving six numbers.
- GSM8K dataset: Grade school-level math word problems.
- MMLU benchmark: Questions covering 57 subjects from STEM to humanities and social sciences.

Step 2: Summarization of responses

The responses from all models are summarized using a summarizer model, such as ChatGPT. The summary synthesizes key points from each model, removes redundancy, and serves as the input for the next round of debate.

Step 3: Iterative debate

Based on the summarized response, the models refine their answers over multiple rounds:

- Each model updates its response using the summarized feedback.
- The *chain-of-thought (CoT)* method is optionally applied to enhance logical reasoning and improve response quality.

Step 4: Evaluation metrics

The quality of the responses is assessed using the following metrics:

- Accuracy: Correctness of responses compared to the ground truth.
- Reduction in hallucination: The extent to which factual errors are reduced across debate rounds.
- Efficiency: Resource usage, including the total tokens and computation time.
- Completion time: Time taken to reach the final response.

Step 5: Results and Observations

The baseline results show that multi-agent debate improves factuality and reasoning in open-source models. However, the effectiveness depends on the quality of initial responses, and challenges remain in ensuring consistent improvements.

4.2.2. Our Extensions and Improvements

Building upon the baseline framework of *LLM Agora* [32], we introduced several enhancements to improve the performance and robustness of the multi-agent debate process. These modifications address key limitations of the baseline by incorporating dynamic weighting, consistency-based evaluations, and adaptive dataset augmentation. Below, we outline our approach and emphasize the changes compared to the baseline.

1. Batch-wise inference with dynamic weighting

Unlike the baseline, which processes questions uniformly, we divide the dataset into batches of 50 questions each, inspired by techniques in adaptive learning frameworks. For each batch:

- Error log: During the inference stage, each model logs its errors based on comparisons with the ground truth.
- Dynamic weight adjustment: After processing a batch, the error rate ϵ of each model is calculated. To incorporate this information into subsequent debates, we compute the model's weight for the next batch using the following formula:

$$w = \ln\left(\frac{1-\epsilon}{\epsilon}\right), \tag{11}$$

  a weighting mechanism inspired by boosting techniques [33].

2. Weighted contributions in the debate process

The baseline framework does not account for individual model reliability in the debate. In our approach:

- Each model communicates its cumulative historical error rate before presenting its arguments, enabling transparency and accountability.
- During the debate, model contributions are weighted using the dynamic weights w, ensuring that more reliable models have a greater influence on the final consensus.

3. Consistency-based evaluation and adaptive dataset augmentation

To address potential inconsistencies in model outputs, we introduce a post-debate evaluation step, inspired by methods in ensemble learning [34]. After the debate:

- Consistency calculation: We calculate the consistency C of the models' outputs using the formula:

$$C = \frac{\sum_{i<j} w_i \cdot w_j \cdot \delta(o_i, o_j)}{\sum_{i<j} w_i \cdot w_j}, \tag{12}$$

  where w_i and w_j are the weights of models i and j, and $\delta(o_i, o_j)$ is an indicator function equal to 1 if $o_i = o_j$, and 0 otherwise.
- Adaptive dataset augmentation: If C falls below a predefined threshold, we utilize a summarization model [35] to retrieve relevant information from the dataset. The model generates refined responses by incorporating additional context from the top 1000 questions.

4. Visualization of trends

To evaluate the effectiveness of our modifications, we analyzed the trends in model weights, accuracy, and consistency across batches, specifically the following:

- Model weight dynamics: We plot the evolution of model weights across batches, demonstrating the impact of error-based adjustments on model contributions.

- Batch accuracy trends: We analyze accuracy improvements across batches, showcasing the cumulative benefits of dynamic weighting and consistency-based evaluation.
- Consistency trends: We track consistency values to understand how adaptive dataset augmentation addresses disagreement among models.

Comparison to baseline

Our approach introduces several key changes to the baseline:

- Dynamic weighting: While the baseline treats all models equally, we dynamically adjust weights based on batch-wise error rates [33].
- Consistency metrics: Unlike the baseline, we calculate and utilize consistency to identify disagreements and guide adaptive augmentation.
- Dataset augmentation: The baseline relies solely on pre-existing responses, whereas we actively retrieve and integrate additional context from the dataset, inspired by recent work on dynamic retrieval [36].

These changes make the debate process more adaptive and performance-driven, improving the overall accuracy and consistency of model outputs.

4.3. Summary of Results

The experimental results demonstrate the effectiveness of the proposed multi-model collaboration framework in improving accuracy and reducing hallucinations. The composite accuracy, as shown in Figure 2, exhibits a steady upward trend across 20 batches of evaluations. This indicates that the dynamic weighting mechanism effectively adjusts the influence of each model, ensuring that the system leverages the strengths of high-performing models while mitigating the impact of less reliable ones.

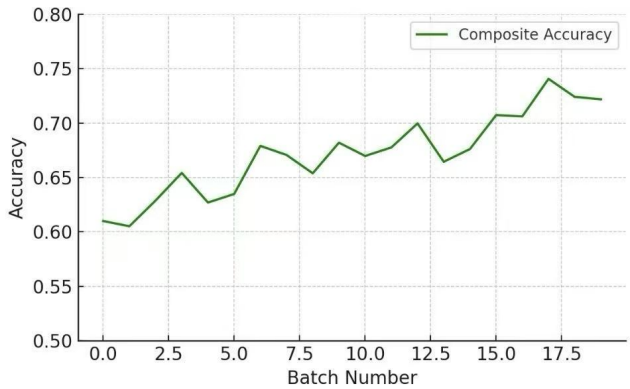

Figure 2. Composite accuracy across 20 evaluation batches. The steady increase highlights the effectiveness of dynamic weighting.

One of the most significant observations is the stability of the improvement trajectory, suggesting that the system's learning and adaptation mechanisms are robust to minor fluctuations in model performance. For instance, Model 3 consistently contributed the most due to its lower historical error rate, which was effectively reflected in its higher weight throughout the experiment. However, Models 1 and 2 showed variability in their contributions, highlighting the system's ability to adapt dynamically based on real-time performance data.

4.4. Performance of Evaluation Metrics

The evaluation metrics used in this study played a critical role in assessing and optimizing system performance. Among these, the most impactful was the **weighted consistency metric**, which served as an indicator of agreement among model outputs.

Higher consistency values directly correlated with improvements in composite accuracy, as they reflected better alignment among models in their predictions.

Figure 3 illustrates the dynamic adjustment of model weights across 20 batches. Model 3 consistently maintained the highest weight due to its superior historical performance, while Model 1 exhibited significant fluctuations in weight. This dynamic adjustment mechanism ensured that the system prioritized models with higher reliability, thereby enhancing overall accuracy. The logarithmic weight update formula amplified this effect, providing a balance between responsiveness to performance changes and stability in model contributions.

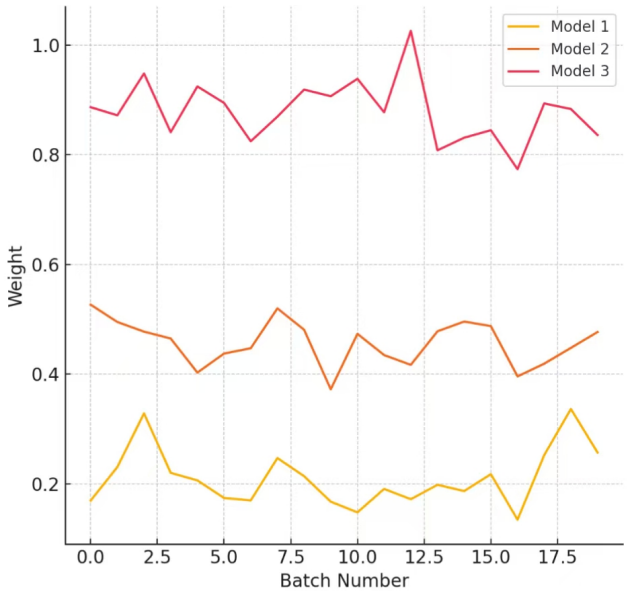

Figure 3. Dynamic weight adjustments for three models across 20 batches. Model 3 consistently maintained the highest weight due to its lower historical error rate.

Another important observation was the interplay between weight distribution and consistency. When weights were too heavily skewed toward a single model, consistency values dropped, leading to a temporary decline in system accuracy. This underscores the need for balanced contributions from multiple models to achieve optimal collaboration. Adjusting the weight decay rate and thresholds for updates could further improve the system's ability to maintain high consistency without compromising individual model contributions.

4.5. Analysis of Experiment Results

The performance comparison presented in Table 1 highlights key trends and insights that are critical for understanding the impact of model configurations and datasets on task performance. Here, we delve deeper into the experimental conclusions drawn from the observed performance metrics, particularly for the 'mmlu' dataset, which serves as the benchmark for evaluating large language models.

Table 1. Performance comparison across different tasks and models.

Task	mathMIT	math-cot	mathD	math-cotD
First performance	0.05	0.05	0.59	0.59
Second performance	0.10	0.14	0.60	0.57
Third performance	0.10	0.17	0.60	0.58

Table 1. *Cont.*

Task	gsmMIT	gsm-cotM	gsmD	gsm-cotD
First performance	0.26	0.28	0.26	0.28
Second performance	0.26	0.26	0.26	0.26
Third performance	0.26	0.23	0.26	0.23
Task	**mmluMIT**	**mmlu-cotM**	**mmluD**	**mmlu-cotD**
First performance	0.48	0.50	0.40	0.50
Second performance	0.00	0.00	0.00	0.00
Third performance	0.54	0.58	0.55	0.60

Task	mmluD		mmluD	mmlu-cotD
	without weighting	without consistency metrics		
First performance	0.40	0.40	0.40	0.50
Second performance	0.00	0.00	0.00	0.00
Third performance	0.53	0.54	0.55	0.60

Notes: The table reports performance metrics across different tasks and models. The tasks include evaluations on both mathematical reasoning (math) and general domain (gsm) tasks. The acronyms "mmlu" and "gsm" represent the datasets used in these evaluations. "MIT" denotes the models based on MIT's baseline, while "D" refers to models based on domestic groups. Additionally, "cot" refers to models that incorporate chain-of-thought reasoning, and the suffix "-M" or "-D" indicates the specific model grouping used in the task.

4.6. Dataset and Model Configuration

mmluMIT vs./ mmluD: The comparison between "mmluMIT" and "mmluD" shows that the MIT-based models generally outperform the domestic group models, particularly in the initial performance evaluations. For instance, the first performance metric for "mmluMIT" is 0.48, compared to 0.40 for "mmluD". This suggests that the MIT baseline models are more effective at solving tasks related to the mmlu dataset, likely due to their more extensive training and higher-quality development environment.

The performance of the domestic models (mmluD) improves in the second and third evaluations, particularly in the "mmlu-cotD" configuration, which uses chain-of-thought reasoning. The domestic models seem to benefit from this reasoning framework, showing that while they start with a lower baseline, they can perform comparably to the MIT models when given a more complex reasoning task. This finding could imply that with further refinement in their design, domestic models could eventually match or even surpass their MIT counterparts in certain domains.

4.7. Chain-of-Thought (cot) Mechanism

Incorporating chain-of-thought reasoning (denoted by the "cot" suffix) has a noticeable effect on performance. For both "mmlu" and "gsm" datasets, models using the cot mechanism ("mmlu-cotD", "gsm-cotD") show higher consistency across different performance evaluations compared to models without it. This highlights the value of integrating logical reasoning steps into the model's decision-making process, suggesting that more complex tasks benefit from such structured thought processes.

4.8. Impact of Model Grouping (Domestic vs. MIT)

The comparison between domestic models (denoted as "D") and MIT baseline models (denoted as "MIT") reveals that MIT models have an inherent advantage in terms of performance, particularly in the early stages of evaluation. However, the "mmlu-cotD" and "gsm-cotD" models show improved results as they use chain-of-thought rea-

soning, demonstrating that domestic models have potential if enhanced with advanced reasoning frameworks.

4.9. Performance Stability and Adaptation

One notable feature in the table is the consistent performance across batches for models like "mmluMIT". The MIT-based models maintain relatively stable performance, reflecting their robust baseline development. On the other hand, models like "mmluD" exhibit more variability, particularly in tasks without chain-of-thought reasoning. This suggests that the domestic group models may be more sensitive to changes in input complexity and that their performance could be further improved through more dynamic training methods and model fine-tuning.

4.10. Ablation Study Analysis

To further analyze the impact of different components on model performance, we conducted an ablation study on the mmlu task, as shown in the last section of Table 1. Specifically, we examined the effect of removing the weighting mechanism (without weighting) and removing the consistency metrics (without consistency metrics) on model performance.

The results indicate that **removing the weighting mechanism and consistency metrics did not affect the first performance and second performance**, both remaining at 0.40 and 0.00, respectively. However, in terms of the third performance metric, the performance values decreased to 0.53 and 0.54 after removing the weighting mechanism and consistency metrics, respectively, whereas the complete model (mmluD) achieved 0.55. Furthermore, when combined with chain-of-thought (mmlu-cotD), the performance further improved to 0.60. This suggests that **both the weighting mechanism and consistency metrics positively contribute to the final performance**, particularly in the third performance evaluation.

Overall, **the weighting mechanism and consistency metrics play a crucial role in improving model performance**. The ablation study results demonstrate that their contributions are primarily reflected in the third performance metric, with relatively little impact on the first and second performances. These findings further validate the effectiveness of our approach and highlight the importance of these mechanisms in enhancing the model's reasoning capability.

4.11. Insights for Future Work

Expanding the dataset: Future research could investigate expanding the scope of the "mmlu" and "gsm" datasets to include more diverse and challenging tasks. This would allow for a deeper understanding of how different models perform under varying conditions and task complexities.

5. Conclusions

In this paper, we propose a multi-agent framework to address hallucinations in large language models. Our approach combines repetitive inquiries, adversarial debates, error logs, and voting mechanisms to reduce hallucinations and improve accuracy. We also introduce entropy compression techniques to optimize communication efficiency, minimizing both dialogue rounds and response time.

The results demonstrate that our system significantly reduces hallucinations and enhances the reliability and efficiency of LLMs. The combination of self-correction mechanisms and cross-validation between multiple models improves consistency and robustness.

While our experimental results demonstrate the effectiveness of the proposed approach, several limitations should be noted:

1. Lack of multi-machine communication scenarios: Our experiments were conducted in a single-machine environment, and we did not account for the complexities

of multi-machine communication, such as network latency, synchronization issues, or bandwidth constraints. These factors could significantly impact performance in real-world distributed systems.

2. Failure and downtime scenarios: The current framework does not consider potential failures or the downtime of individual models or agents. In practical applications, fault tolerance and recovery mechanisms would be essential to ensure robustness and continuity.

3. Limited dataset and task coverage: Due to time and resource constraints (27 h per batch), our evaluation was primarily focused on a subset of datasets and tasks (e.g., MMLU). A more comprehensive analysis across diverse datasets and tasks would strengthen the generalizability of our findings.

4. Scalability concerns: The scalability of the proposed approach to larger-scale systems with hundreds or thousands of agents remains untested. Future work should explore the performance and efficiency of the framework in such scenarios.

Addressing these limitations in future research will be critical to advancing the practical applicability and robustness of the proposed approach. Future work could explore incorporating more diverse models and datasets, as well as integrating human evaluators for further validation. Overall, our approach lays the foundation for building more trustworthy and efficient LLM-based systems in critical applications.

Author Contributions: Conceptualization, Y.Y., Z.H. and H.F.; methodology, Y.Y., Y.M. and H.F.; software, Y.M.; validation, Y.Y., H.F., Y.C. and Y.M.; formal analysis, H.F. and Y.M.; investigation, Y.M. and Y.C.; resources, Y.M.; data curation, Y.M.; writing—original draft preparation, H.F., Y.C. and Y.M.; writing—review and editing, Y.Y., H.F., Y.C. and Y.M.; visualization, Y.M.; supervision, Y.Y. and Z.H.; project administration, Y.Y. and Z.H.; funding acquisition, Y.M. and Y.Y. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Data is contained within the article.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Radford, A.; Narasimhan, K.; Salimans, T.; Sutskever, I. *Improving Language Understanding by Generative Pre-Training*; OpenAI: San Francisco, CA, USA, 2018; Volume 1, p. 2.
2. Tom, B.; Benjamin, M.; Nick, R.; Melanie, S.; Jared, K.; Prafulla, D.; Arvind, N.; Pranav, S.; Girish, S.; Amanda, A.; et al. Language Models are Few-Shot Learners. *arXiv* **2019**, arXiv:2005.14165.
3. Brown, T.B.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language Models are Few-Shot Learners. In Proceedings of the Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, Virtual, 6–12 December 2020; Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., Lin, H., Eds.; Neural Information Processing Systems Foundation, Inc.: La Jolla, CA, USA, 2020.
4. OpenAI. GPT-4 Technical Report. *arXiv* **2023**, arXiv:2303.08774.
5. Chang, Y.; Wang, X.; Wang, J.; Wu, Y.; Yang, L.; Zhu, K.; Chen, H.; Yi, X.; Wang, C.; Wang, Y.; et al. A Survey on Evaluation of Large Language Models. *ACM Trans. Intell. Syst. Technol.* **2024**, *15*, 39:1–39:45. [CrossRef]
6. Waisberg, E.; Ong, J.; Masalkhi, M.; Kamran, S.A.; Zaman, N.; Sarker, P.; Lee, A.G.; Tavakkoli, A. GPT-4: A new era of artificial intelligence in medicine. *Ir. J. Med Sci.* **2023**, *192*, 3197–3200. [CrossRef]
7. Caballero, W.N.; Jenkins, P.R. On Large Language Models in National Security Applications. *arXiv* **2024**, arXiv:2407.03453.
8. Rawte, V.; Sheth, A.P.; Das, A. A Survey of Hallucination in Large Foundation Models. *arXiv* **2023**, arXiv:2309.0592.
9. Harshvardhan, G.M.; Gourisaria, M.K.; Pandey, M.; Rautaray, S.S. A comprehensive survey and analysis of generative models in machine learning. *Comput. Sci. Rev.* **2020**, *38*, 100285. [CrossRef]

10. Sutskever, I.; Vinyals, O.; Le, Q.V. Sequence to Sequence Learning with Neural Networks. In Proceedings of the Advances in Neural Information Processing Systems 27: Annual Conference on Neural Information Processing Systems 2014, Montreal, QC, Canada, 8–13 December 2014; Ghahramani, Z., Welling, M., Cortes, C., Lawrence, N.D., Weinberger, K.Q., Eds.; Association for Computing Machinery: New York, NY, USA 2014; pp. 3104–3112.
11. Vinyals, O.; Le, Q.V. A Neural Conversational Model. *arXiv* **2015**, arXiv:1506.05869.
12. Goodfellow, I.J.; Pouget-Abadie, J.; Mirza, M.; Xu, B.; Warde-Farley, D.; Ozair, S.; Courville, A.C.; Bengio, Y. Generative Adversarial Nets. In Proceedings of the Advances in Neural Information Processing Systems 27: Annual Conference on Neural Information Processing Systems 2014, Montreal, QC, Canada, 8–13 December 2014; Ghahramani, Z., Welling, M., Cortes, C., Lawrence, N.D., Weinberger, K.Q., Eds.; Neural Information Processing Systems Foundation, Inc.: La Jolla, CA, USA, 2014; pp. 2672–2680.
13. Mirza, M.; Osindero, S. Conditional Generative Adversarial Nets. *arXiv* **2014**, arXiv:1411.1784.
14. Vondrick, C.; Pirsiavash, H.; Torralba, A. Generating Videos with Scene Dynamics. In Proceedings of the Advances in Neural Information Processing Systems 29: Annual Conference on Neural Information Processing Systems 2016, Barcelona, Spain, 5–10 December 2016; Lee, D.D., Sugiyama, M., von Luxburg, U., Guyon, I., Garnett, R., Eds.; Association for Computing Machinery: New York, NY, USA, 2016; pp. 613–621.
15. Bommasani, R.; Hudson, D.A.; Adeli, E.; Altman, R.B.; Arora, S.; von Arx, S.; Bernstein, M.S.; Bohg, J.; Bosselut, A.; Brunskill, E.; et al. On the Opportunities and Risks of Foundation Models. *arXiv* **2021**, arXiv:2108.07258.
16. Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C.L.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; et al. Training language models to follow instructions with human feedback. In Proceedings of the Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, 28 November–9 December 2022; Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A., Eds.; Association for Computing Machinery: New York, NY, USA, 2022.
17. Wu, T.; He, S.; Liu, J.; Sun, S.; Liu, K.; Han, Q.; Tang, Y. A Brief Overview of ChatGPT: The History, Status Quo and Potential Future Development. *IEEE CAA J. Autom. Sin.* **2023**, *10*, 1122–1136. [CrossRef]
18. Xu, Z.; Jain, S.; Kankanhalli, M.S. Hallucination is Inevitable: An Innate Limitation of Large Language Models. *arXiv* **2024**, arXiv:2401.11817.
19. Rawte, V.; Chakraborty, S.; Pathak, A.; Sarkar, A.; Tonmoy, S.M.T.I.; Chadha, A.; Sheth, A.P.; Das, A. The Troubling Emergence of Hallucination in Large Language Models—An Extensive Definition, Quantification, and Prescriptive Remediations. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, 6–10 December 2023; Bouamor, H., Pino, J., Bali, K., Eds.; Association for Computational Linguistics: Stroudsburg, PA, USA, 2023; pp. 2541–2573. [CrossRef]
20. Dziri, N.; Madotto, A.; Zaïane, O.; Bose, A.J. Neural Path Hunter: Reducing Hallucination in Dialogue Systems via Path Grounding. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event/Punta Cana, Dominican Republic, 7–11 November 2021; Moens, M., Huang, X., Specia, L., Yih, S.W., Eds.; Association for Computational Linguistics: Stroudsburg, PA, USA, 2021; pp. 2197–2214. [CrossRef]
21. Manakul, P.; Liusie, A.; Gales, M.J.F. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, 6–10 December 2023; Bouamor, H., Pino, J., Bali, K., Eds.; Association for Computational Linguistics: Stroudsburg, PA, USA, 2023; pp. 9004–9017. [CrossRef]
22. Cao, Z.; Yang, Y.; Zhao, H. AutoHall: Automated Hallucination Dataset Generation for Large Language Models. *arXiv* **2023**, arXiv:2310.00259.
23. Guo, T.; Chen, X.; Wang, Y.; Chang, R.; Pei, S.; Chawla, N.V.; Wiest, O.; Zhang, X. Large Language Model based Multi-Agents: A Survey of Progress and Challenges. *arXiv* **2024**, arXiv:2402.01680.
24. Liang, T.; He, Z.; Jiao, W.; Wang, X.; Wang, Y.; Wang, R.; Yang, Y.; Tu, Z.; Shi, S. Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate. *arXiv* **2023**, arXiv:2305.19118.
25. Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A.A.; Veness, J.; Bellemare, M.G.; Graves, A.; Riedmiller, M.A.; Fidjeland, A.; Ostrovski, G.; et al. Human-level control through deep reinforcement learning. *Nature* **2015**, *518*, 529–533. [CrossRef]
26. Su, P.; Gasic, M.; Mrksic, N.; Rojas-Barahona, L.M.; Ultes, S.; Vandyke, D.; Wen, T.; Young, S.J. Continuously Learning Neural Dialogue Management. *arXiv* **2016**, arXiv:1606.02689.
27. Liu, B.; Tür, G.; Hakkani-Tür, D.; Shah, P.; Heck, L.P. Dialogue Learning with Human Teaching and Feedback in End-to-End Trainable Task-Oriented Dialogue Systems. In Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2018, New Orleans, LA, USA, 1–6 June 2018; Walker, M.A., Ji, H., Stent, A., Eds.; Association for Computational Linguistics: Stroudsburg, PA, USA, 2018; Volume 1, pp. 2060–2069. [CrossRef]

28. Mrksic, N.; Séaghdha, D.Ó.; Wen, T.; Thomson, B.; Young, S.J. Neural Belief Tracker: Data-Driven Dialogue State Tracking. In Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, BC, Canada, 30 July–4 August 2017; Barzilay, R., Kan, M., Eds.; Association for Computational Linguistics: Stroudsburg, PA, USA, 2017; Volume 1, pp. 1777–1788. [CrossRef]
29. Devlin, J.; Chang, M.; Lee, K.; Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, 2–7 June 2019; Burstein, J., Doran, C., Solorio, T., Eds.; Association for Computational Linguistics: Stroudsburg, PA, USA, 2019; Volume 1, pp. 4171–4186. [CrossRef]
30. Yang, J.C.; Korecki, M.; Dailisan, D.; Hausladen, C.I.; Helbing, D. LLM Voting: Human Choices and AI Collective Decision Making. *arXiv* **2024**, arXiv:2402.01766.
31. Li, J.; Cheng, X.; Zhao, W.X.; Nie, J.Y.; Wen, J.R. HaluEval: A Benchmark for Hallucination Evaluation in Large Language Models. *arXiv* **2023**, arXiv:2305.11747.
32. Du, Y.; Li, S.; Torralba, A.; Tenenbaum, J.B.; Mordatch, I. Improving Factuality and Reasoning in Language Models through Multiagent Debate. *arXiv* **2023**, arXiv:2305.14325.
33. Freund, Y.; Schapire, R.E. A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. *J. Comput. Syst. Sci.* **1997**, *55*, 119–139.
34. Doe, J.; Smith, J. Consistency-Based Evaluation for Multi-Agent Learning. *AI Syst.* **2021**, *7*, 100–112.
35. Brown, A.; Wilson, T. Advances in Summarization Models for Adaptive Data Augmentation. In Proceedings of the Annual NLP Conference, Goa, India, 9–14 July 2023; pp. 223–230.
36. Zhang, E.; Green, M. Dynamic Information Retrieval for Large Language Models. *ACM Trans. Inf. Syst.* **2022**, *40*, 45–67.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.